ROS 2 Service & Client

Complete Tutorial

Fahrenheit to Celsius Converter

A step-by-step guide from package creation to running nodes

What you will build:

A ROS 2 service that accepts a Fahrenheit value,
converts it to Celsius, and returns a status message.

Environment

Item	Version
ROS 2	Humble Hawksbill
OS	Ubuntu 22.04 LTS
Python	3.10+
Build Tool	colcon

Contents

1 Service vs Topic — Why Services Exist **3**
1.1 Comparison Table . . . 3
1.2 Mental Model . . . 3

2 What We Are Building **3**

3 The Service Type **3**

4 Create the Packages **4**
4.1 Step 1 — Navigate to the workspace source folder . . . 4
4.2 Step 2 — Create the interface package . . . 4
4.3 Step 3 — Create the Python node package . . . 4
4.4 Step 4 — Final workspace structure . . . 5

5 Create the Service Definition **5**
5.1 Step 1 — Create the srv folder and file . . . 5
5.2 Step 2 — Write the .srv content . . . 5
5.3 Step 3 — Update CMakeLists.txt . . . 5
5.4 Step 4 — Update my_temp_interfaces/package.xml . . . 6
5.5 Step 5 — Build and verify the interface . . . 6

6 Update the Converter Package **7**
6.1 Update my_temp_converter/package.xml . . . 7

7 Write the Service Server **7**
7.1 Create the file . . . 7
7.2 Write the server code . . . 7
7.3 Key Lines Explained . . . 9

8 Write the Service Client **9**
8.1 Create the file . . . 9
8.2 Write the client code . . . 9
8.3 Key Lines Explained . . . 11

9 Register Entry Points in setup.py **11**
9.1 Entry Point Syntax . . . 12

10 Final Package Structure **13**

11 Build Everything **13**

12 Run the Service **14**
12.1 Terminal 1 — Start the server . . . 14
12.2 Terminal 2 — Run the client . . . 14
12.3 Sample Output — Body Temperature . . . 14
12.4 Reference Values . . . 15

13 Test with the ROS 2 CLI **15**

14 The Conversion Formula **15**
14.1 Status Thresholds 16

15 Complete Communication Flow **16**

16 Common Mistakes & Fixes **17**

17 Quick Reference **17**
17.1 Commands Cheat Sheet 17
17.2 Server vs Client — Side by Side 17

1 Service vs Topic — Why Services Exist

Before writing any code, it is important to understand **when to use a service instead of a topic**.

1.1 Comparison Table

	Topic	Service
Pattern	Publish / Subscribe	Request / Response
Direction	One way	Two way
Response	No response	Always gets a response
Relationship	One to many	One client to one server
Timing	Continuous / streaming	On demand / one shot
Example use	Lidar data, camera feed	Convert temperature, trigger action

1.2 Mental Model

```
Topic   -->  like a radio broadcast
             publisher talks, subscribers listen, no reply ever

Service -->  like a phone call
             client calls, server picks up, server replies, call ends
```

2 What We Are Building

A simple **temperature conversion service**:

- Client sends one float: `fahrenheit`
- Server converts it using the formula $C = \frac{(F - 32) \times 5}{9}$
- Server returns: `celsius`, `status`, and `message`
- Client prints the formatted result

```
Client                        Server
------                        ------
sends: fahrenheit=98.6  --->  converts: (98.6 - 32) * 5/9 = 37.0
receives: celsius=37.0  <---  returns: celsius=37.0
         status=HOT               status="HOT"
         message="..."            message="98.6 F is hot. Stay hydrated!"
```

3 The Service Type

Every ROS 2 service type is defined in a `.srv` file. The file is split by `--` into two halves.

FahrenheitToCelsius.srv

```
# Request -- what the client sends
float64 fahrenheit
---
# Response -- what the server sends back
float64 celsius
string  status
string  message
```

Key Rule

Everything **above** `--` is the **request** (filled by the client).
Everything **below** `--` is the **response** (filled by the server).

4 Create the Packages

We need **two packages**:

1. `my_temp_interfaces` — ament_cmake package that holds the `.srv` file
2. `my_temp_converter` — ament_python package that holds the server and client nodes

4.1 Step 1 — Navigate to the workspace source folder

```
cd ~/ros2_ws/src
```

4.2 Step 2 — Create the interface package

```
ros2 pkg create my_temp_interfaces \
  --build-type ament_cmake \
  --dependencies rosidl_default_generators
```

Argument	Purpose
`my_temp_interfaces`	Package name
`-build-type ament_cmake`	Required for custom `.srv` files
`-dependencies rosidl_default_generators`	Generates Python/C++ from `.srv`

4.3 Step 3 — Create the Python node package

```
ros2 pkg create my_temp_converter \
  --build-type ament_python \
  --dependencies rclpy
```

4.4 Step 4 — Final workspace structure

```
ros2_ws/
 src/
     my_temp_interfaces/       <- ament_cmake (srv definitions)
        CMakeLists.txt
        package.xml

     my_temp_converter/        <- ament_python (nodes)
         my_temp_converter/
            __init__.py
         package.xml
         setup.cfg
         setup.py
         resource/
             my_temp_converter
```

5 Create the Service Definition

5.1 Step 1 — Create the srv folder and file

```
mkdir my_temp_interfaces/srv
touch my_temp_interfaces/srv/FahrenheitToCelsius.srv
```

5.2 Step 2 — Write the .srv content

Open `FahrenheitToCelsius.srv` and write:

```
float64 fahrenheit
---
float64 celsius
string  status
string  message
```

5.3 Step 3 — Update CMakeLists.txt

Replace the entire content of `my_temp_interfaces/CMakeLists.txt` with:

Listing 1: my_temp_interfaces/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(my_temp_interfaces)

find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
```

```
  "srv/FahrenheitToCelsius.srv"
)

ament_package()
```

Common Mistake

The filename in `CMakeLists.txt` must **exactly match** the actual `.srv` filename — including spelling and capitalisation. A typo here will cause a build failure.

5.4 Step 4 — Update my_temp_interfaces/package.xml

Listing 2: my_temp_interfaces/package.xml

```
<?xml version="1.0"?>
<package format="3">
  <name>my_temp_interfaces</name>
  <version>0.0.0</version>
  <description>Custom service interfaces for temperature</
     description>
  <maintainer email="you@email.com">your_name</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <build_depend>rosidl_default_generators</build_depend>
  <exec_depend>rosidl_default_runtime</exec_depend>
  <member_of_group>rosidl_interface_packages</member_of_group>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

5.5 Step 5 — Build and verify the interface

```
cd ~/ros2_ws
colcon build --packages-select my_temp_interfaces
source install/setup.bash
ros2 interface show my_temp_interfaces/srv/FahrenheitToCelsius
```

Expected Output

```
float64 fahrenheit
---
float64 celsius
string status
string message
```

Always Source After Building

After every `colcon build`, always run `source install/setup.bash` in the **same terminal** before using any `ros2` commands. Without this, ROS 2 cannot find your new package.

6 Update the Converter Package

6.1 Update my_temp_converter/package.xml

Listing 3: my_temp_converter/package.xml

```
<?xml version="1.0"?>
<package format="3">
  <name>my_temp_converter</name>
  <version>0.0.0</version>
  <description>Fahrenheit to Celsius converter ROS 2 service</
     description>
  <maintainer email="you@email.com">your_name</maintainer>
  <license>Apache-2.0</license>

  <exec_depend>rclpy</exec_depend>
  <exec_depend>my_temp_interfaces</exec_depend>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>pytest</test_depend>

  <export>
    <build_type>ament_python</build_type>
  </export>
</package>
```

The key addition is `<exec_depend>my_temp_interfaces</exec_depend>` — this tells ROS 2 that our converter depends on our custom interface package.

7 Write the Service Server

7.1 Create the file

```
touch ~/ros2_ws/src/my_temp_converter/my_temp_converter/
   fahrenheit_server.py
```

7.2 Write the server code

```
import rclpy
from rclpy.node import Node
from my_temp_interfaces.srv import FahrenheitToCelsius
```

```python


class FahrenheitServer(Node):

    def __init__(self):
        super().__init__('fahrenheit_server')

        self.srv = self.create_service(
            FahrenheitToCelsius,        # service type
            'fahrenheit_to_celsius',    # service name
            self.convert_callback       # callback function
        )

        self.get_logger().info('Fahrenheit to Celsius Server is ready...')

    def convert_callback(self, request, response):

        # Core Conversion Formula: C = (F - 32) x 5/9
        response.celsius = (request.fahrenheit - 32.0) * 5.0 / 9.0

        # Determine status based on the celsius value
        if response.celsius <= 0.0:
            response.status  = 'FREEZING'
            response.message = (
                f'{request.fahrenheit}F is at or below freezing point.'
            )
        elif response.celsius < 15.0:
            response.status  = 'COLD'
            response.message = (
                f'{request.fahrenheit}F is cold. Wear a jacket!'
            )
        elif response.celsius < 25.0:
            response.status  = 'WARM'
            response.message = (
                f'{request.fahrenheit}F is pleasantly warm.'
            )
        elif response.celsius < 100.0:
            response.status  = 'HOT'
            response.message = (
                f'{request.fahrenheit}F is hot. Stay hydrated!'
            )
        else:
            response.status  = 'BOILING'
            response.message = (
                f'{request.fahrenheit}F is at or above boiling point.'
            )

```

```python
        self.get_logger().info(
            f'{request.fahrenheit}F  -->  {response.celsius:.2f}C'
            f'  [{response.status}]'
        )

        return response     # ALWAYS return the response


def main(args=None):
    rclpy.init(args=args)
    node = FahrenheitServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Listing 4: fahrenheit_server.py

7.3 Key Lines Explained

Code	Meaning
`create_service(Type, name, callback)`	Registers the service with ROS 2
`def convert_callback(self, req, res)`	Called automatically when client sends a request
`response.celsius = ...`	You fill in the response fields
`return response`	Critical — client waits forever if omitted
`rclpy.spin(node)`	Keeps server alive, waiting for requests

Never Forget This

You **must** always `return response` at the end of your callback. If you forget, the client will hang indefinitely waiting for a reply that never comes.

8 Write the Service Client

8.1 Create the file

```
touch ~/ros2_ws/src/my_temp_converter/my_temp_converter/
    fahrenheit_client.py
```

8.2 Write the client code

```python
import sys
import rclpy
from rclpy.node import Node
from my_temp_interfaces.srv import FahrenheitToCelsius


class FahrenheitClient(Node):

    def __init__(self):
        super().__init__('fahrenheit_client')

        self.client = self.create_client(
            FahrenheitToCelsius,        # must match server type
            'fahrenheit_to_celsius'     # must match server name
        )

        # Block until the server is available
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('Waiting for server...')

        self.get_logger().info('Server found! Ready to convert.')

    def send_request(self, fahrenheit):
        request = FahrenheitToCelsius.Request()
        request.fahrenheit = fahrenheit     # fill in the value

        # Send request without blocking -- returns a Future
        self.future = self.client.call_async(request)

        # Wait here until the Future has a result
        rclpy.spin_until_future_complete(self, self.future)

        # Extract and return the response from the Future
        return self.future.result()


def main(args=None):
    rclpy.init(args=args)
    node = FahrenheitClient()

    # Read the fahrenheit value from the command line
    # Usage: ros2 run my_temp_converter fahrenheit_client 98.6
    fahrenheit = float(sys.argv[1])

    result = node.send_request(fahrenheit)

    node.get_logger().info('================================')
    node.get_logger().info('    TEMPERATURE CONVERSION      ')
    node.get_logger().info('================================')
    node.get_logger().info(f'  Input    : {fahrenheit} F')
    node.get_logger().info(f'  Output   : {result.celsius:.2f} C')
```

```
    node.get_logger().info(f'  Status  : {result.status}')
    node.get_logger().info(f'  Message : {result.message}')
    node.get_logger().info('================================')

    node.destroy_node()
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Listing 5: fahrenheit_client.py

8.3 Key Lines Explained

Code	Meaning
`create_client(Type, name)`	Creates a client (2 args, no callback)
`wait_for_service(timeout_sec=1.0)`	Waits up to 1s for server; loops if not found
`FahrenheitToCelsius.Request()`	Creates an empty request object
`call_async(request)`	Sends request without blocking; returns Future
`spin_until_future_complete(...)`	Blocks until the server replies
`future.result()`	Extracts the actual response from the Future
`float(sys.argv[1])`	Reads first command-line argument as float

Common Mistake

Always pass a number when running the client:

```
# WRONG -- no number given, crashes with IndexError
ros2 run my_temp_converter fahrenheit_client

# CORRECT
ros2 run my_temp_converter fahrenheit_client 98.6
```

9 Register Entry Points in setup.py

Open `my_temp_converter/setup.py` and update it:

```
from setuptools import find_packages, setup

package_name = 'my_temp_converter'

setup(
    name=package_name,
    version='0.0.0',
```

```
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='your_name',
    maintainer_email='you@email.com',
    description='Fahrenheit to Celsius ROS 2 service',
    license='Apache-2.0',
    extras_require={
        'test': ['pytest'],
    },
    entry_points={
        'console_scripts': [
            'fahrenheit_server = my_temp_converter.
               fahrenheit_server:main',
            'fahrenheit_client = my_temp_converter.
               fahrenheit_client:main',
        ],
    },
)
```

Listing 6: setup.py

9.1 Entry Point Syntax

```
'command_name = package.module:function'
 ------------   ----------------------
 What you type  Where Python looks for it
```

Command	Package	Module	Function
`fahrenheit_server`	`my_temp_converter`	`fahrenheit_server.py`	`main()`
`fahrenheit_client`	`my_temp_converter`	`fahrenheit_client.py`	`main()`

10 Final Package Structure

```
ros2_ws/
 src/
     my_temp_interfaces/          <- ament_cmake (srv definitions)
        srv/
           FahrenheitToCelsius.srv
        CMakeLists.txt
        package.xml

     my_temp_converter/           <- ament_python (nodes)
         my_temp_converter/
            __init__.py
            fahrenheit_server.py
            fahrenheit_client.py
         package.xml
         setup.cfg
         setup.py
         resource/
             my_temp_converter
```

11 Build Everything

Build Order Matters

You **must** build `my_temp_interfaces` **before** `my_temp_converter` — because the converter depends on the interfaces.

```
cd ~/ros2_ws

# Step 1 -- build the interface package first
colcon build --packages-select my_temp_interfaces

# Step 2 -- source so the new interfaces are available
source install/setup.bash

# Step 3 -- build the converter package
colcon build --packages-select my_temp_converter

# Step 4 -- source again
source install/setup.bash
```

Expected Build Output

```
Starting >>> my_temp_interfaces
Finished <<< my_temp_interfaces [6.62s]
Summary: 1 package finished [7.03s]
```

```
Starting >>> my_temp_converter
Finished <<< my_temp_converter [1.73s]
Summary: 1 package finished [2.18s]
```

12 Run the Service

12.1 Terminal 1 — Start the server

```
source ~/ros2_ws/install/setup.bash
ros2 run my_temp_converter fahrenheit_server
```

Server Output

```
[INFO] [fahrenheit_server]: Fahrenheit to Celsius Server is ready...
```

12.2 Terminal 2 — Run the client

Test multiple values:

```
source ~/ros2_ws/install/setup.bash

# Freezing point
ros2 run my_temp_converter fahrenheit_client 32.0

# Body temperature
ros2 run my_temp_converter fahrenheit_client 98.6

# Boiling point
ros2 run my_temp_converter fahrenheit_client 212.0

# Room temperature
ros2 run my_temp_converter fahrenheit_client 72.0
```

12.3 Sample Output — Body Temperature

Client Output: fahrenheit_client 98.6

```
[INFO] [fahrenheit_client]: ================================
[INFO] [fahrenheit_client]:     TEMPERATURE CONVERSION
[INFO] [fahrenheit_client]: ================================
[INFO] [fahrenheit_client]:   Input   : 98.6 F
[INFO] [fahrenheit_client]:   Output  : 37.00 C
[INFO] [fahrenheit_client]:   Status  : HOT
[INFO] [fahrenheit_client]:   Message : 98.6F is hot. Stay hydrated!
[INFO] [fahrenheit_client]: ================================
```

12.4 Reference Values

Input (°F)	Output (°C)	Status	Description
32.0	0.00	FREEZING	Freezing point of water
50.0	10.00	COLD	Cold day
72.0	22.22	WARM	Room temperature
98.6	37.00	HOT	Body temperature
212.0	100.00	BOILING	Boiling point of water

13 Test with the ROS 2 CLI

You can call the service **without a client node** using the CLI:

```
# Call directly from terminal
ros2 service call /fahrenheit_to_celsius \
  my_temp_interfaces/srv/FahrenheitToCelsius \
  "{fahrenheit: 77.0}"

# List all active services
ros2 service list

# Check the service type
ros2 service type /fahrenheit_to_celsius
```

CLI Output

```
response:
  my_temp_interfaces.srv.FahrenheitToCelsius_Response(
    celsius=25.0,
    status='WARM',
    message='77.0F is pleasantly warm.'
  )
```

14 The Conversion Formula

$$C = \frac{(F - 32) \times 5}{9}$$

Input (°F)	Calculation	Result (°C)
32	$(32 - 32) \times 5/9$	0.00
98.6	$(98.6 - 32) \times 5/9$	37.00
212	$(212 - 32) \times 5/9$	100.00
77	$(77 - 32) \times 5/9$	25.00

14.1 Status Thresholds

Celsius Range	Status
$C \leq 0$	FREEZING
$0 < C < 15$	COLD
$15 \leq C < 25$	WARM
$25 \leq C < 100$	HOT
$C \geq 100$	BOILING

15 Complete Communication Flow

```
Terminal command:
ros2 run my_temp_converter fahrenheit_client 98.6
          |
          v
main()
  rclpy.init()
  node = FahrenheitClient()
    create_client('fahrenheit_to_celsius')
    wait_for_service() -- server found
          |
          v
  fahrenheit = float(sys.argv[1])  -->  98.6
          |
          v
  send_request(98.6)
    request.fahrenheit = 98.6
    call_async(request)  ------------------------>  FahrenheitServer
    spin_until_future_complete()                      convert_callback()
    future.result()  <------------------------          celsius = 37.0
          |                                             status  = 'HOT'
          v                                             message = '...'
  result.celsius  = 37.0                                return response
  result.status   = 'HOT'
  result.message  = '98.6F is hot...'
          |
          v
  prints formatted report
  destroy_node()
  rclpy.shutdown()
  exits
```

16 Common Mistakes & Fixes

Mistake	Error	Fix
Forgot to pass a number to client	`IndexError: list index`	`fahrenheit_client 98.6`
Forgot `return response`	Client hangs indefinitely	Always `return response` in callback
Typo in `CMakeLists.txt` filename	CMake build error	Ensure exact match with `.srv` filename
Forgot to source after build	`Package not found`	`source install/setup.bash`
Wrong service name in client	Client waits forever	Must match server exactly
Build order wrong	Import error	Build interfaces before converter
Server not running	Service not available	Start server in Terminal 1 first

17 Quick Reference

17.1 Commands Cheat Sheet

```
# Build a specific package
colcon build --packages-select <package_name>

# Source the workspace
source install/setup.bash

# Run a node
ros2 run <package> <node>

# List all services
ros2 service list

# Call a service from terminal
ros2 service call /<service_name> <type> "<args>"

# Show service type definition
ros2 interface show <package>/srv/<ServiceName>
```

17.2 Server vs Client — Side by Side

Server	**Client**
`create_service(Type, name, callback)`	`create_client(Type, name)`
Callback receives request + response	Calls `wait_for_service()`
Fills response fields	Creates `Request()` and fills it
Returns response	Calls `call_async(request)`
Runs forever with `rclpy.spin()`	Waits with `spin_until_future_complete()`
Stays alive	Exits after one request

End of Tutorial — ROS 2 Fahrenheit to Celsius Service & Client